perplexity

I would like to understand this lecture material better for my deep learning class. This lecture is on graph neural networks, can you please review the lecture notes and give me a broad overview of the material covered for me to understand

Welcome! What concept do you want to learn more about? Some options based on this lecture are:

- How CNNs can be viewed as GNNs on image grids
- Message passing and permutation-invariant aggregation on graphs
- Design of a basic GNN layer (f, g, similarity, expectation)
- Normalization and residuals in GNNs
- Relation between GNNs and Transformers

First, at a high level, here's what this lecture is doing:

1. **Connect CNNs → GNNs**
    - Treat an image as a graph: pixels = nodes, edges connect neighbors (e.g., 3×3 neighborhood).
    - A 3×3 convolution = each node receives messages from its neighbors via *edge-type-specific* weight matrices (N, S, E, W, NE, ...) plus a self-node term and bias.
    - This shows CNNs are just a special case of GNNs on a regular grid with fixed edge labels.
2. **Generalize from grids to arbitrary graphs**
    - In images, the grid provides structure; in GNNs, the *input graph* provides that structure.
    - Weight sharing still holds: every node applies the same update rule.
    - Normalization becomes tricky because graphs in a batch can have different numbers of nodes; so you often normalize per node embedding instead of across spatial dimensions like in CNNs.
3. **Undirected / unlabelled graphs and permutation invariance**
    - For undirected edges or unlabeled edges, you can't rely on fixed labels like N/E/S/W.
    - Each node has:
        - its own embedding h_0,

- the number of neighbors n,
- a *set* $\{h_1, \ldots, h_n\}$ of neighbor embeddings.

- The aggregation over neighbors must be **permutation-invariant**: sum, mean, max, etc.
- General form:

$$\text{new } h_0 = f_{w_1}\Big(h_0, \sum_i g_{w_2}(h_i)\Big)$$

or more generally:

$$f_{w_1}\Big(h_0, \sum_i s_{w_2}(h_0, h_i)\, g_{w_3}(h_i)\Big)$$

where:

- g transforms neighbor features,
- s scores neighbors (similarity/attention),
- the sum (or expectation-like operator) aggregates,
- f combines with the node's own embedding.

4. **A concrete GNN building block**
   - Design choices suggested in the notes:
     - **g**: a learned linear transform on neighbor features.
     - **s**: inner product–style similarity between node and neighbor (like attention scores).
     - Aggregation: expectation-style (normalized weighted sum).
     - **f**: an MLP on the concatenation of self-embedding and aggregated neighbor message.
   - Residual/skips + mild nonlinearities help avoid oversmoothing and allow deeper stacks of these layers.
5. **Tasks: graph, node, edge prediction**
   - **Graph classification**: aggregate node embeddings into a single graph embedding, then classify.
   - **Node classification**: use each node's embedding to predict a node label.
   - **Edge prediction**: predict whether an edge exists between two nodes based on their embeddings.
6. **Connection to Transformers**
   - If you take a **complete graph** (every node connected to every other), and use attention-style s and weighted sums, you essentially recover a Transformer layer.
   - So Transformers can be seen as GNNs on a fully connected graph over tokens.

Before we go deeper into any of these, what are your learning goals with this material (e.g., doing well on exams, implementing a GNN for research, understanding theory connections to Transformers)?

⁂

<q>First, at a high level, here’s what this lecture is doing:

Connect CNNs → GNNs

Treat an image as a graph: pixels = nodes, edges connect neighbors (e.g., 3×3 neighborhood).

A 3×3 convolution = each node receives messages from its neighbors via edge-type-specific weight matrices (N, S, E, W, NE, ...) plus a self-node term and bias.

This shows CNNs are just a special case of GNNs on a regular grid with fixed edge labels.

Generalize from grids to arbitrary graphs

In images, the grid provides structure; in GNNs, the input graph provides that structure.

Weight sharing still holds: every node applies the same update rule.

Normalization becomes tricky because graphs in a batch can have different numbers of nodes; so you often normalize per node embedding instead of across spatial dimensions like in CNNs. </q> So does the entire graph get input to the GNN? In a CNN, the "graph" is the kernel, but this get's ran over an entire image. For a GNN, if the "graph" is the kernel, then what is the image?

The entire graph is the input to the GNN, not just a “kernel” like in CNNs. In a CNN, the kernel (a small filter) slides over the image, operating on local patches, but the full image is processed by repeatedly applying this kernel. In contrast, a GNN operates directly on the full graph structure: all nodes and edges are processed simultaneously, with each node updating its representation based on information from its neighbors in the graph.[2] [3] [4]

So, for a GNN:

- The "image" is replaced by the graph itself (nodes = pixels, edges = connections between pixels).
- The "kernel" concept is replaced by the GNN layer, which performs message passing: each node gathers messages from its neighbors, aggregates them, and updates its own state.[5] [2]
- This process is repeated for all nodes in the graph, and for multiple layers, allowing information to propagate throughout the graph.

Unlike CNNs, which use a fixed-size kernel and slide it over the image, GNNs use the graph's connectivity to determine which nodes interact, so the entire graph is always present as input, and the GNN layers transform the node embeddings while preserving the graph structure.[3] [4] [2]

⁂

I see, so each node receives information from its neighbors like in a CNN kernel. So if we have a graph with nodes and edges, does that mean that each node has the same function applied to it? I mean, if each node receives information from its neighbors, does that pass through the same aggregation and activation function? How does that work?

Yes, that's exactly the right intuition, and you're asking the key question.

One concept for this step: shared node update rule

In a standard message-passing GNN layer:

1. **Same functions for all nodes**
   - Every node uses the **same** message, aggregation, and update functions.
   - Formally, for each node v:
     $$m_{u \to v} = g_\theta(h_u) M_v = \mathrm{AGG\_} u \in \mathcal{N}(v) m_{u \to v} h'_v = f_\phi(h_v, M_v)$$
   - g_θ, AGG, and f_ϕ are **shared** across all nodes (like a CNN kernel is shared across locations).
2. **Differences between nodes**
   - Nodes end up with *different* outputs because:
     - They start with different features h_v.
     - They have different neighborhoods $\mathcal{N}(v)$.
   - So same code/parameters, but different inputs → different results.

This weight sharing is what makes the layer **permutation-equivariant** over nodes and efficient to train.

To check your understanding:
Can you restate in your own words *why* we want to share the same g, AGG, f across all nodes, instead of giving each node its own separate parameters?

⁂

If we tried to force a specific graph topology into the network that would make it not very useful, so instead we want to learn a function which works for many graph topologies. The way we do this is by finding a symmetry over all topologies; one symmetry that's easy to notice is all topologies have nodes with edges. Therefore if we learn a function which, for each node, collects information across its edges and aggregates that information to transform its state, this architecture could work for many different topologies and be generally useful. Is that right?

You're very close, and your intuition is good. Let's tighten it a bit.

One concept: local symmetry = "same rule everywhere"

- We *don't* hard-code a specific graph topology into the parameters.
- Instead, we assume a **symmetry**: if you relabel (permute) the nodes, the computation should just relabel accordingly.
- To respect this, every node uses the **same update rule**:

$$h_v' = f\big(h_v,\ \mathrm{AGG}_{u \in \mathcal{N}(v)} g(h_u)\big)$$

- AGG (sum/mean/max) is **permutation-invariant**, so the result doesn't depend on neighbor order and works on any topology.

So yes: by learning a *shared* "collect-from-neighbors + aggregate + update" rule, the network can be applied to many different graphs without changing its parameters.

Quick check: in your own words, why must the aggregation over neighbors be permutation-invariant for this to work across arbitrary graphs?

⁂

Because graph topology is variable based on problems

Nice start. Let's sharpen that a bit.

One key idea: neighbors have no fixed order

In a graph, a node's neighbors form a **set**, not a sequence:

- There is no canonical "first neighbor, second neighbor, …".
- If we arbitrarily reorder neighbors, the *meaning* of the graph shouldn't change.

So, if the GNN is to work on *any* graph:

- The aggregation over neighbors must give the **same result** no matter how neighbors are ordered.
- That's why we use permutation-invariant ops like sum / mean / max.

So: it's not just that topology varies; it's that neighbor *ordering is meaningless*.

Question: can you name one permutation-invariant aggregator and one that is *not* permutation-invariant?

The mean is a permutation invariant aggregator, the first value of the sequence is not

Nice, you've got the right idea but let's clean up the wording.

- **Correct example (permutation-invariant):**
  - Mean of neighbor embeddings (also sum, max). Reordering inputs doesn't change the result.
- **Incorrect example (not permutation-invariant):**
  - "Take the first value in the sequence" (or any operation that depends on index/order).

So a better phrasing:
"The **mean** is permutation-invariant; **taking the first element** of the neighbor list is not."

We've now nailed permutation-invariant aggregation (Step 1 of understanding message passing).
Next: do you want to dig into how the *message function* g and *update function* f are typically parameterized (MLPs, linear layers, attention)?

⁂

Yes let's dig into that

Great. Since your goal is to understand the lecture, here's our **lesson plan** and how it helps:

- **Step 1**: Message & update functions (what are g and f concretely?)
- **Step 2**: Adding similarity/attention $s(h_v, h_u)$
- **Step 3**: Layer stacking, residuals, and oversmoothing
- **Step 4**: Special cases: GCN vs GAT vs Transformer-as-GNN

- **Step 5**: How this maps to implementations (PyTorch-style)

This directly supports your goal of intuitively understanding GNN layers beyond notation.

Step 1: What are g and f in practice?

Take a simple message-passing layer:

$$m_{u \to v} = g_\theta(h_u), \quad M_v = \mathrm{AGG}_{u \in \mathcal{N}(v)} m_{u \to v}, \quad h'_v = f_\phi(h_v, M_v)$$

Typical choices:

1. **Message function** g_θ
   - Usually a **shared linear layer** or small MLP:
     $$g_\theta(h_u) = W_g h_u + b_g$$
   - Same W_g for all edges; sometimes depends on edge features too.
2. **Update function** f_ϕ
   - Combines self and aggregated neighbor info, often via concat + MLP:
     $$h'_v = \sigma\big(W_f [h_v \,\|\, M_v] + b_f\big)$$
     where σ is a nonlinearity (ReLU, GELU, etc.).

So the whole layer is basically:
“Linear/MLP on neighbors → sum/mean → MLP with self + neighbors.”

Question: in your own words, what’s the conceptual role difference between g and f in this setup?

Conceptually, g is a parameterized function which transforms features from other notes before aggregating with the self node; g applies independently and identically to each other edge node (although it may not be identical if the g function is additionally dependent on edge features). Therefore, it is the message function, meaning messages from other nodes pass through g. f, on the other hand, is applied after aggregation and updates the current nodes features based on its previous state and the messages passed from all neighbors.

A question I have here is that it appears only f has an activation function, is this always the case? Why does g not also have an activation function?

Your description of **g** and **f** is exactly right conceptually.

- **g**: transforms each neighbor's features into a "message," applied (usually) identically across edges, possibly using edge features.
- **f**: takes the *aggregated* messages plus the node's own features and produces the updated node state.

On your question:

> it appears only f has an activation function, is this always the case? Why does g not also have an activation function?

Not always. Both are design choices:

- You can have:
  - **Linear g, nonlinear f** (very common; simple messages, all nonlinearity in the update).
  - **Nonlinear g and nonlinear f** (e.g., both are MLPs with ReLU).
  - Even **only nonlinear g, linear f**, etc.

The lecture is showing a *minimal* expressive block; putting the nonlinearity in **f** is often enough for universal approximation when you stack layers, so they keep **g** simple (linear) to reduce parameters and keep analysis clean.

Modern GNNs (GraphSAGE, GAT, etc.) often have nonlinearities in both the message and the update parts—think "MLP before aggregate, MLP after aggregate."

Quick check: if you *removed all* activation functions from both g and f in a deep GNN, what do you think would happen to the kind of functions the network can represent?

⁂

The network would now have no source of nonlinearity, and would functionally be just a sequence of matrix multiplications, and therefore would be a very complex set of linear transformations and much less expressive

Exactly. Nicely put.

- With no nonlinearities, each layer is linear in the inputs.
- Composition of linear maps is still linear, so a deep GNN without activations collapses to "one big" linear map on the initial node features → very limited expressivity.

So far in our lesson plan, you've got:

- shared update rule,
- permutation-invariant aggregation,
- roles of g (messages) and f (updates),
- why we need nonlinearities.

Next single concept: **similarity/attention** $s(h_v, h_u)$ inside message passing.

At a high level, why might it be useful for a node to *weight* different neighbors differently instead of just averaging them?

Certain neighbors may be more important than others. Intuitively, this can be thought of as distance or similarity, where the more close/similar two nodes are the more likely the information from one is to contain something useful for the other. Therefore, adding a similarity measure will help the network weight more relevant information

Your reasoning is exactly on target.

One concept for this step: **similarity / attention weights**

We modify the aggregation to:

$$h'_v = f\Big(h_v, \sum_{u \in \mathcal{N}(v)} s(h_v, h_u)\, g(h_u)\Big)$$

- $g(h_u)$: the raw message from neighbor u.
- $s(h_v, h_u)$: a *learned* scalar weight (often normalized like softmax) measuring how relevant u is to v.
- The sum is now a **weighted** combination: important neighbors contribute more; noisy/irrelevant ones are downweighted.

Your "distance/similarity → more useful information" story is exactly the idea behind GATs and Transformer-style attention.

To see this concretely: would you like to walk through how $s(h_v, h_u)$ is parameterized in a Graph Attention Network (GAT)?

⁂

Yes please do

Great, this moves us to the "attention/similarity" part of the lesson plan, which directly supports your goal of understanding how GNNs connect to Transformers.

We'll do **one concept**: how GAT defines $s(h_v, h_u)$.

GAT-style attention

For each edge $(u \to v)$:

1. **Linear projection of node features**
$$z_v = W h_v, \quad z_u = W h_u$$
Same W for all nodes.
2. **Compute an unnormalized attention score**
$$e_{vu} = \mathrm{LeakyReLU}\big(a^\top [z_v \,\|\, z_u]\big)$$
   - a is a learned vector.
   - $[\cdot \| \cdot]$ is concatenation.
3. **Normalize across neighbors with softmax**
$$\alpha_{vu} = \mathrm{softmax}_u(e_{vu}) = \frac{\exp(e_{vu})}{\sum_{k \in \mathcal{N}(v)} \exp(e_{vk})}$$
This α_{vu} is your $s(h_v, h_u)$.
4. **Weighted message aggregation**
$$h'_v = \sigma\Big(\sum_{u \in \mathcal{N}(v)} \alpha_{vu} z_u\Big)$$

So GAT's similarity function is:
"Project nodes → concatenate → linear score → LeakyReLU → softmax."

Check your understanding: in your own words, why does the softmax over neighbors make this look very similar to Transformer attention?

The softmax over neighbors makes this look similar to attention because it forces the weights to look like attention scores/probabilities, where their output sums to one. Therefore, the neighbors have relative attention scores. to each other. If one neighbor gets a very high weight relative to the rest, its attention score is very close to one and the rest are low. Likewise, if two scores have the same high weight, their score is the same and attention is split between them

Exactly—great explanation.

You've now connected:

- **GNN message passing**
  $h'_v = f\big(h_v,\ \sum_u s(h_v, h_u)\, g(h_u)\big)$
- **GAT / Transformer-style attention**
  where $s(h_v, h_u)$ is a softmax-normalized score over neighbors, giving a probability distribution over which neighbors to "listen" to most.

Softmax ⇒ weights sum to 1 ⇒ the new node embedding is a **convex combination** of projected neighbor features, just like a token attends over other tokens in a Transformer.

So far in our lesson plan we've covered:

- shared node update rule & permutation invariance,
- roles of g (messages) and f (update),
- need for nonlinearities,
- attention-style similarity s (GAT-like).

Next, to round out the big picture from the lecture, we should hit **one more concept**: how stacking many such layers leads to *information propagation* and issues like **oversmoothing**, and why residuals/normalization are used.

Before that: how would you describe, in your own words, what information a node's embedding after *k* GNN layers contains compared to after 1 layer?

After k GNN layers, the nodes has received k messages passed from its neighbors and aggregated it in itself. Therefore, it has mixed with its neighbors k times

Nice and concise; let's sharpen that idea a bit.

One concept: k-hop neighborhood

After 1 GNN layer:

- $h_v^{(1)}$ depends on v and its **1-hop neighbors**.

After 2 layers:

- $h_v^{(2)}$ depends on neighbors' neighbors ⇒ **2-hop neighborhood**.

After k layers:

- $h_v^{(k)}$ contains information from all nodes within **k hops** in the graph (plus its own history), because each layer propagates information one step further.

So it's not just "mixed k times," it's "mixed with all nodes reachable within k steps."

This leads to **oversmoothing**: deep GNNs can make node embeddings within a connected region become very similar.

Question: why might oversmoothing be a problem for node classification tasks?

Right! That makes sense, each neighbor receives information from its neighbors also at each step. Therefore, after k steps, we have a receptive field of k steps in all directions away from the node

Exactly, that's a great way to phrase it: a **k-hop receptive field** over the graph.

We're near the end of our lesson plan; quick new concept: **oversmoothing problem**.

Because each layer averages / mixes neighbors:

- As depth increases, node embeddings within the same connected region tend to **converge**.
- Eventually, many nodes look almost the same in feature space.
- For node classification, this makes it **hard to distinguish** nodes with different labels if they're in the same region.

This is one reason for:

- residual/skip connections,

- normalization,
- and sometimes limiting depth or using tricks like jumping knowledge.

To wrap this up: do you want to next (a) connect this formally to Transformers-as-GNNs, or (b) talk about normalization/residuals in GNN layers?

Let me get this clear; over smoothing is because the neighbors average their neighbors information? Therefore, after many k (layers), the aggregation converges closer to just a local average value, causing the network to be smooth. What if we want to have information pass between distal nodes in the network?

Yes, that's the right core picture.

- Each layer does a kind of **Laplacian smoothing**: mix a node with its neighbors.
- Repeating this k times ≈ repeatedly applying a smoothing operator.
- As $k \to \infty$, node features in a connected region converge to a **local average** (or stationary distribution) → **oversmoothing**: everyone looks similar, little discrimination left.

Now your second question: *how to pass information between distal nodes without oversmoothing?*
Key idea: we want **long-range dependence** without doing naïve "average neighbors" too many times.

Common strategies (conceptual, not an exhaustive list):

1. **Skip / residual connections**
   Keep a copy of earlier, less-smoothed representations:
   $$h_v^{(k+1)} = \alpha\, h_v^{(k)} + (1-\alpha)\,\mathrm{GNNLayer}(h^{(k)})$$
   or even jump from layer 0: $h_v^{(k+1)} = h_v^{(0)} + \mathrm{GNNLayer}(h^{(k)})$.
   Deep stack → long-range info; residuals → preserve individuality.
2. **"Teleport" / global connections**
   - Add edges that connect distant nodes (e.g., via learned or heuristic edges).
   - Or use **Transformer-style full attention** on nodes: every node attends to all others in one step.
     This passes distal information without many local averaging steps.
3. **Personalized PageRank / diffusion with reset**
   Mix local propagation with a reset to the original features (PPR, APPNP, etc.):
   $$H^{(\infty)} = \alpha(I - (1-\alpha)\tilde{A})^{-1}X$$
   Intuition: you do many propagations but always "pull back" to initial features, slowing oversmoothing.

4. **Change aggregation / sparsify graph**
    - Less aggressive averaging (e.g., attention that sharply focuses on a few neighbors).
    - Prune dense regions so mixing is slower there.

Your intuition: “more layers → averaging → convergence to local average” is exactly what theory formalizes via spectral analysis and random walks.

To check understanding: if you wanted distal nodes to share information **without** destroying local distinctions, which of the strategies above sounds most appealing to you, and why?

⁂

Global / teleport connections sound like a reasonable solution because they directly address the issue; they allow for information to pass between large distances without causing over smoothing

That’s a very good instinct.

Global / teleport connections (including full attention) let you:

- **Reach far nodes in 1–2 layers**, instead of many local-averaging steps.
- Thus you **separate “distance” from “depth”**:
    - Depth can focus more on *computation* / feature transformation,
    - while graph structure (including long edges) handles *who can talk to whom*.

This is exactly why you can view a **Transformer as a GNN on a fully connected graph**:
each token/node attends to every other in a single layer, so long-range interaction is cheap, and oversmoothing is controlled by attention patterns instead of many diffusion steps.

We’ve now covered the main conceptual pieces from your lecture:

1. CNNs as GNNs on grids
2. Shared node update rule and permutation-invariant aggregation
3. Message function g, update function f, and nonlinearities
4. Attention/similarity $s(h_v, h_u)$ (GAT, Transformer-like)
5. k-hop receptive fields and oversmoothing, plus ideas to handle long-range info

Well done—this is basically the core mental model of GNN layers.

Would you like to (a) connect all this explicitly to the Transformer block equations, or (b) shift to implementation-level questions (e.g., batching graphs, PyTorch Geometric / DGL style)?

I want to cover normalization; how do we perform normalization on graphs?

Normalization on graphs is trickier than in CNNs because graphs vary in size, degree, and batch structure. Let's add **one concept** at a time.

First, clarify your goal: are you mainly trying to (a) understand what your lecture means by "normalize per node embedding instead of across spatial dims," or (b) survey the main normalization types used in GNNs (BatchNorm, LayerNorm, GraphNorm, etc.)?

To get started: what do you already know about BatchNorm and LayerNorm in standard (non-graph) deep nets?

⁂

I do know about batch and layer norm , I want to understand (a)

Good, then let's focus on (a): what your lecture means by "normalize per node embedding instead of across spatial dims."

We'll do **one concept**: how CNN vs GNN normalization axes differ.

CNN: what are we normalizing over?

Take BatchNorm for a convolution layer with output $X \in \mathbb{R}^{B\times C\times H\times W}$:

- B: batch,
- C: channels,
- H, W: spatial (image) dims.

BatchNorm usually computes mean/var **per channel**, over **batch and spatial positions**:

$$\mu_c = \frac{1}{BHW}\sum_{b,h,w} X_{b,c,h,w}$$

and normalizes each $X_{b,c,h,w}$ using μ_c, σ_c.

Key: the spatial grid is **fixed and regular**; you can safely average across all locations for a channel.

GNN: what's different?

In a GNN layer, output is usually shaped like:

- **Single graph**: $H \in \mathbb{R}^{N\times d}$
- **Batch of graphs**: concatenate nodes ⇒ $H \in \mathbb{R}^{(\sum_g N_g)\times d}$

Here:

- N = number of nodes (varies per graph),
- d = feature dimension.

There is **no fixed 2D grid**, and different graphs have **different numbers of nodes**. So "spatial dims" don't exist in the same way.

When the notes say *"normalize per node embedding instead of across spatial dimensions"*, they mean:

- In CNN: normalize per channel, aggregating over all spatial positions (and batch).
- In GNN: common choice is to normalize **within each node's feature vector**, *not* across nodes.

Concretely, two common patterns:

1. **LayerNorm-style (per node)**
   For each node v, embedding $h_v \in \mathbb{R}^d$:
   - Compute mean/var over its **features**:
     $$\mu_v = \frac{1}{d}\sum_{j=1}^{d} h_{v,j}, \quad \sigma_v^2 = \frac{1}{d}\sum_{j}(h_{v,j}-\mu_v)^2$$
   - Normalize:
     $$\hat{h}_{v,j} = \gamma_j \frac{h_{v,j}-\mu_v}{\sigma_v} + \beta_j$$

   This is analogous to **LayerNorm** in Transformers: each token/node normalized independently over its features.
2. **BatchNorm-style (per feature across nodes)**
   You *can* do BatchNorm by treating nodes as the "instances":
   - For feature dimension j, compute mean/var over all nodes in batch:
     $$\mu_j = \frac{1}{\#\text{nodes}}\sum_{v} h_{v,j}$$
   - Normalize $h_{v,j}$ with μ_j, σ_j.

   But now "instances" = nodes, not images; graphs of different sizes contribute different numbers of nodes, which can introduce **batch noise** and imbalance. This is why GNN-specific norms like GraphNorm/NodeNorm/GroupNorm have been proposed.

The lecture's message is: because graphs vary in size and have no regular spatial structure, **it's more natural to normalize at the level of each node embedding (or each graph) rather than over spatial dimensions** like in CNNs.

A concrete mental mapping for you:

- CNN BatchNorm: normalize per channel over (batch × spatial positions).
- Transformer LayerNorm: normalize per token over features.

- GNN common practice (in line with lecture): normalize per **node** over features (LayerNorm-like), or per **graph** over nodes (GraphNorm).

Check: in your own words, what does “per node embedding normalization” mean in terms of which axis we’re taking mean/variance over?

⁂

per node embedding normalization would be to normalize each node by computing the mean/variance over all channels and normalizing all channels by those values

Your query is exactly right.

Per **node embedding normalization** means:

- For each node v with embedding $h_v \in \mathbb{R}^d$,
- You compute the **mean and variance over its channels/features**:

$$\mu_v = \frac{1}{d}\sum_{j=1}^{d} h_{v,j}, \quad \sigma_v^2 = \frac{1}{d}\sum_{j=1}^{d}(h_{v,j} - \mu_v)^2$$

- Then you normalize **all channels of that node** using μ_v, σ_v.

That’s exactly analogous to LayerNorm done “per token” in Transformers, but here it’s “per node.”

To check: how would you contrast this with a BatchNorm-style normalization over nodes in a batch?

⁂

You can't do batch norm on graphs, but you can do graph norm where you compute normalization statistics over all nodes in a single graph and normalize each graph by its own nodes

Not quite; you *can* do BatchNorm on graphs, but it’s often **not ideal**, which is why methods like GraphNorm were proposed.

Your core idea about **GraphNorm** is right though:

- BatchNorm for GNNs:
  Treat **nodes as batch elements** and, for each feature dimension j, compute mean/var over **all nodes in all graphs** in the minibatch, then normalize each node’s $h_{v,j}$ using those stats. [102] [103]

This is allowed, but the batch stats can be very noisy because graphs in a batch vary a lot in size/structure.[103] [102]

- GraphNorm:
  For each **graph separately**, compute mean/var over **its own nodes**, and normalize its nodes using *graph-specific* stats (sometimes with a learnable shift).[104] [102]
  This is closer to what you said: "compute normalization statistics over all nodes in a single graph and normalize each graph by its own nodes."

So the correction is:

- You **can** do BatchNorm on graphs (and people do),
- but GraphNorm instead normalizes **per graph**, using only that graph's nodes.

To check understanding: can you restate the *main drawback* of BatchNorm when applied naively to GNNs on diverse graphs?

⁂

The main drawbacks of BatchNorm are that each graph has potentially very different numbers of nodes, and so if we compute batch stats over all graphs/nodes it may affect some graphs very differently than others

Yes, that's the key issue.

Because BatchNorm for GNNs would:

- Treat **all nodes from all graphs in the minibatch** as a single pool when computing mean/variance per feature,
- Large graphs contribute many more nodes than small graphs,
- Graphs can have very different scales / distributions of features,

the batch statistics can be dominated by a few large or atypical graphs, making normalization noisy or harmful for others.

That's why people consider per-node (LayerNorm-style) or per-graph (GraphNorm) normalization instead.

Given all this, do you feel clear on normalization now, or would you like to see how a full GNN layer would be written with attention + normalization + residuals, similar to a Transformer block?

1. Lecture-12-GNN.pdf
2. https://distill.pub/2021/gnn-intro
3. https://www.ibm.com/think/topics/graph-neural-network
4. https://viso.ai/deep-learning/graph-neural-networks/

5. https://www.geeksforgeeks.org/deep-learning/what-are-graph-neural-networks/
6. https://mlu-explain.github.io/neural-networks/
7. https://www.sciencedirect.com/science/article/pii/S2666651021000012
8. https://icpmconference.org/2021/wp-content/uploads/sites/5/2021/09/Process-Discovery-using-Graph-Neural-Networks.pdf
9. https://en.wikipedia.org/wiki/Graph_neural_network
10. https://pmc.ncbi.nlm.nih.gov/articles/PMC8360394/
11. https://www.nature.com/articles/s43246-022-00315-6
12. https://distill.pub/2021/gnn-intro
13. https://pmc.ncbi.nlm.nih.gov/articles/PMC8360394/
14. https://en.wikipedia.org/wiki/Graph_neural_network
15. https://viso.ai/deep-learning/graph-neural-networks/
16. https://www.sciencedirect.com/science/article/pii/S2666651021000012
17. https://www.ibm.com/think/topics/graph-neural-network
18. https://www.geeksforgeeks.org/deep-learning/what-are-graph-neural-networks/
19. https://neptune.ai/blog/graph-neural-network-and-some-of-gnn-applications
20. https://www.datacamp.com/tutorial/comprehensive-introduction-graph-neural-networks-gnns-tutorial
21. https://www.reddit.com/r/MachineLearning/comments/fs5j1f/p_an_illustrated_guide_to_graph_neural_networks/
22. https://arxiv.org/abs/2411.19392
23. http://arxiv.org/pdf/1910.03802.pdf
24. https://distill.pub/2021/gnn-intro
25. http://papers.neurips.cc/paper/9675-understanding-the-representation-power-of-graph-neural-networks-in-learning-graph-topology.pdf
26. https://liner.com/review/graph-neural-networks-for-edge-signals-orientation-equivariance-and-invariance
27. https://openreview.net/forum?id=UqrFPhcmFp
28. https://www.youtube.com/watch?v=CznfqG9Aigo
29. https://www.sciencedirect.com/science/article/pii/S266682702400063X
30. https://ui.adsabs.harvard.edu/abs/2022APS..MARM09002B/abstract
31. https://arxiv.org/abs/1812.09902
32. https://proceedings.mlr.press/v162/huang22l/huang22l.pdf
33. https://www.cs.mcgill.ca/~wlh/grl_book/files/GRL_Book-Chapter_5-GNNs.pdf
34. https://arxiv.org/abs/2205.14368
35. https://www.reddit.com/r/MachineLearning/comments/1jctc0p/d_any_new_interesting_methods_to_represent/
36. https://www.sciencedirect.com/science/article/pii/S1063520325000521
37. https://apxml.com/courses/introduction-to-graph-neural-networks/chapter-2-the-message-passing-mechanism/permutation-invariance-equivariance
38. https://arxiv.org/pdf/2205.14368.pdf

39. https://icml.cc/media/icml-2022/Slides/16659_urBfEKz.pdf
40. https://iclr-blogposts.github.io/2023/blog/2023/sets-and-graphs/
41. https://openreview.net/forum?id=gELzFTYQDi
42. https://distill.pub/2021/gnn-intro
43. https://www.heptalytics.ai/graph-neural-networks-gnn
44. https://arxiv.org/abs/2307.04661
45. https://en.wikipedia.org/wiki/Graph_neural_network
46. https://www.ibm.com/think/topics/graph-neural-network
47. https://scikit-network.readthedocs.io/en/master/reference/gnn.html
48. https://www.v7labs.com/blog/neural-networks-activation-functions
49. https://visual-intelligence-umn.github.io/GNN-101/
50. https://karthick.ai/blog/2024/Graph-Neural-Network/
51. https://www.sciencedirect.com/science/article/abs/pii/S0893608021000757
52. https://pmc.ncbi.nlm.nih.gov/articles/PMC7416665/
53. https://www.ibm.com/think/topics/attention-mechanism
54. https://arxiv.org/html/2403.15419v1
55. https://epichka.com/blog/2023/gat-paper-explained/
56. https://file.cz123.top/3PhD/1EGG/Conference4EGG/[33]19 Understanding attention and generalization in graph neural networks.pdf
57. https://www.baeldung.com/cs/graph-attention-networks
58. https://petar-v.com/GAT/
59. https://cs.stanford.edu/people/jure/pubs/magna-ijcai21.pdf
60. https://arxiv.org/abs/1710.10903
61. https://www.reddit.com/r/MachineLearning/comments/1jgwjjk/d_are_gnns_obsolete_because_of_transformers/
62. https://arxiv.org/abs/2303.10993
63. https://openreview.net/forum?id=QqG3wI3c8L
64. https://disco.ethz.ch/courses/fs21/seminar/talks/GNN_Oversmoothing.pdf
65. https://www.youtube.com/watch?v=MLiEoJOhXJA
66. https://openreview.net/forum?id=Kg65qieiuB&noteId=sSqhiAOpsV
67. https://arxiv.org/html/2407.11928v1
68. https://apxml.com/courses/graph-neural-networks-gnns/chapter-3-gnn-training-complexities/mitigating-oversmoothing
69. https://papers.nips.cc/paper_files/paper/2023/file/6e4cdfdd909ea4e34bfc85a12774cba0-Paper-Conference.pdf
70. https://proceedings.neurips.cc/paper_files/paper/2023/file/6e4cdfdd909ea4e34bfc85a12774cba0-Paper-Conference.pdf
71. https://www.reddit.com/r/MachineLearning/comments/tqdpf8/d_why_gnns_suffer_from_oversmoothing_but_cnns_dont/
72. https://arxiv.org/abs/2009.11746

73. https://proceedings.nips.cc/paper/2020/file/33dd6dba1d56e826aac1cbf23cdcca87-Paper.pdf
74. https://www.sciencedirect.com/science/article/abs/pii/S0925231222000030
75. https://discuss.dgl.ai/t/graph-node-edge-normalization-techniques/3080
76. http://proceedings.mlr.press/v139/cai21e/cai21e.pdf
77. https://arxiv.org/html/2404.13344v2
78. https://www.microsoft.com/en-us/research/publication/graphnorm-a-principled-approach-to-accelerating-graph-neural-network-training/
79. https://openreview.net/forum?id=reEp2BReEou
80. https://neurips.cc/virtual/2024/poster/93483
81. https://openreview.net/forum?id=oLltLS5F9R
82. https://www.sciencedirect.com/science/article/abs/pii/S0925231222000030
83. https://cs.mcgill.ca/~wlh/comp766/files/chapter4_draft_mar29.pdf
84. https://discuss.dgl.ai/t/graph-node-edge-normalization-techniques/3080
85. https://arxiv.org/abs/2009.11746
86. https://arxiv.org/pdf/2103.00164.pdf
87. https://proceedings.nips.cc/paper/2020/file/33dd6dba1d56e826aac1cbf23cdcca87-Paper.pdf
88. https://www.microsoft.com/en-us/research/publication/graphnorm-a-principled-approach-to-accelerating-graph-neural-network-training/
89. http://proceedings.mlr.press/v139/cai21e/cai21e.pdf
90. https://cs.mcgill.ca/~wlh/comp766/files/chapter4_draft_partial_feb12.pdf
91. https://www.sciencedirect.com/science/article/pii/S0020025523003493
92. https://discuss.dgl.ai/t/graph-node-edge-normalization-techniques/3080
93. https://cs.mcgill.ca/~wlh/comp766/files/chapter4_draft_mar29.pdf
94. https://www.sciencedirect.com/science/article/abs/pii/S0925231222000030
95. https://arxiv.org/pdf/2103.00164.pdf
96. https://arxiv.org/abs/2009.11746
97. https://openreview.net/pdf?id=reEp2BReEou
98. https://www.microsoft.com/en-us/research/publication/graphnorm-a-principled-approach-to-accelerating-graph-neural-network-training/
99. http://proceedings.mlr.press/v139/cai21e/cai21e.pdf
100. https://cs.mcgill.ca/~wlh/comp766/files/chapter4_draft_partial_feb12.pdf
101. https://www.sciencedirect.com/science/article/pii/S0020025523003493
102. http://proceedings.mlr.press/v139/cai21e/cai21e.pdf
103. https://arxiv.org/abs/2009.03294
104. https://openreview.net/forum?id=reEp2BReEou
105. https://stackoverflow.com/questions/45463778/instance-normalisation-vs-batch-normalisation/52627872
106. https://www.reddit.com/r/MachineLearning/comments/5i940e/d_how_does_batch_normalization_not_completely/
107. https://gradientscience.org/batchnorm/

108. http://d2l.ai/chapter_convolutional-modern/batch-norm.html
109. https://en.wikipedia.org/wiki/Batch_normalization
110. https://www.youtube.com/watch?v=hdMlYbqyzJQ
111. https://www.sciencedirect.com/science/article/abs/pii/S0925231222000030